

Electrical and Computer Engineering
University of Thessaly (UTH)

ECE - Parallel Computer Architecture

Spring Semester — Educational year 2024-2025

Project

Machine Learning Branch Prediction

Spyridon Liaskonis - AEM: 03381

Georgios Kapakos - AEM: 03165

Contents

1	Introduction	4
2	Branch Prediction Techniques	4
2.1	Static Prediction Strategies	4
2.1.1	Backward Taken, Forward Not Taken (BTFNT)	4
2.1.2	Profile-Guided Optimization (PGO)	4
2.2	Dynamic Prediction	5
2.2.1	1-bit predictor	5
2.2.2	Bimodal predictor	5
2.3	Advanced Dynamic Prediction	6
2.3.1	Two-Level Adaptive Predictors	6
2.4	State-of-the-Art: TAGE-SC-L	7
2.4.1	TAGE (TAgged GEometric history length)	7
2.4.2	Statistical Corrector (SC)	8
2.4.3	Loop Predictor (L)	8
2.5	Machine/Deep Learning Architectures	9
2.5.1	Perceptron Predictor	9
2.5.2	Tarsa’s Predictor	9
2.5.3	BranchNet	10
2.6	Our Architecture: <i>BranchNet-LSTM</i>	10
2.6.1	Input Sequences	10
2.6.2	Embedding	11
2.6.3	Conv1D	11
2.6.4	AvgPool1D	11
2.6.5	LSTM	12
2.6.6	Fully Connected Dense Layers and Sigmoid Activation	12
3	HLS4ML	14
3.1	Challenges	14
3.1.1	Model Complexity	14
3.1.2	Framework Compatibility	14
3.1.3	Memory Constraints	14
3.1.4	Training without Batch size	14
3.2	Process	14
3.2.1	Model Training & Evaluation	14
3.2.2	Quantization Configuration	14
3.2.3	Model Conversion	14
3.2.4	Model Compilation	15
3.2.5	Model Building	15
3.2.6	Performance Reporting	15
3.2.7	Bitstream Generation	15
4	Dataset Generation	15
4.1	Computer Architecture Simulators	15
4.1.1	ChampSim	15
4.1.2	Traces	16
4.2	Hard-to-Predict (H2P) Branches	16

4.2.1	Identifying H2P branches	16
4.3	Dataset Structure	17
5	Model Training	17
5.1	Training Data	17
6	Model Evaluation	17
6.1	Results for different predictors	18
6.2	Comparing our model with the Original BranchNet	18
7	Future Work	19
8	References	20

1 Introduction

Recent advancements in branch prediction architectures have been made using traditional methods. However, the growing complexity of modern processors has increased the need for more sophisticated approaches, such as the integration of neural networks. These neural networks leverage data from previous executions to pre-train models, which are then deployed as part of the branch prediction unit. This approach is particularly effective in addressing Hard-to-Predict (H2P) branches, which present significant challenges for traditional prediction techniques. The fundamental concept behind this method is to train the model offline and subsequently embed it into the hardware, enhancing the accuracy and efficiency of branch prediction.

The challenge of predicting H2P branches has driven the development of neural network architectures specifically designed to tackle this issue. These branches, despite being few in number, lead to a disproportionate number of mispredictions when using conventional predictors like TAGE-SC-L. Our focus is to develop neural network-based solutions that can improve prediction accuracy for these critical branches.

2 Branch Prediction Techniques

Before Machine Learning was possible, algorithmic branch prediction techniques were applied and still are to this point, dominating the branch prediction mechanism of the CPUs.

2.1 Static Prediction Strategies

Static branch prediction techniques make predictions based on fixed rules or information available at compile time, without relying on the dynamic execution history of the program.

The main advantage of static prediction is its simplicity and low hardware cost, as it requires no runtime state tracking. However, its primary drawback is its inability to adapt to the dynamic behavior of the program during execution. Branch behavior can change based on input data or program phase, which static predictors cannot account for. This inherent limitation, prioritizing low cost over runtime adaptability motivated the development of dynamic prediction techniques.

2.1.1 Backward Taken, Forward Not Taken (BTFNT)

This strategy predicts that branches whose target address is lower than the current program counter (PC) typically indicating a loop will be taken, while branches with a forward target address will not be taken. This leverages the common behavior of loops, which usually iterate multiple times.

2.1.2 Profile-Guided Optimization (PGO)

Allows the compiler to analyze execution profiles from previous runs of the program to determine the likely direction of each branch. This information can then be used by the compiler to provide hints to the hardware or to optimize the code layout itself. Compilers might also employ heuristics based on static code analysis (examining opcodes, operand types, detecting loop structures) to infer likely branch directions.

2.2 Dynamic Prediction

Dynamic branch predictors overcome the limitations of static approaches by utilizing information gathered during runtime about the past behavior of branches. The core assumption is that the recent history of a branch's execution is indicative of its future behavior. These predictors typically employ a hardware structure called a Branch History Table (BHT), which functions like a cache indexed by the lower bits of the branch instruction's PC.¹ Each entry in the BHT stores state information representing the recent history of the corresponding branch.

2.2.1 1-bit predictor

Each BHT entry contains a single bit indicating whether the branch was taken or not taken the last time it was executed. The prediction for the current instance is simply the value stored in that bit. If a misprediction occurs, the bit is flipped. While simple, this scheme performs poorly for branches that frequently alternate direction or for typical loop structures. A loop that executes N times will be mispredicted twice with a 1-bit predictor: once on the first iteration (if the previous execution exited the loop) and once on the final iteration when the loop terminates, resulting in an accuracy of $(N-2)/N$. Overall accuracy is typically around 65-70%.

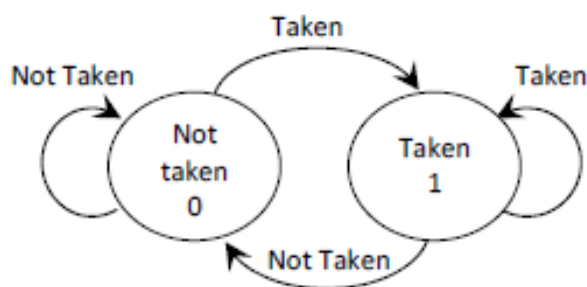


Figure 1: 1-bit predictor

2.2.2 Bimodal predictor

Each BHT entry holds a 2-bit counter, which implements a four-state finite state machine: Strongly Not Taken (00), Weakly Not Taken (01), Weakly Taken (10), and Strongly Taken (11). When a branch is taken, the counter increments; when not taken, it decrements. The counters "saturate" at the ends, meaning they stay at 00 even if decremented further, and stay at 11 even if incremented further. The prediction is typically based on the most significant bit (0 for Not Taken, 1 for Taken). The crucial advantage of the 2-bit counter is hysteresis: a branch must be mispredicted twice consecutively in the same direction to change a "strong" prediction (00 or 11). This prevents the predictor from changing its prediction based on a single, potentially anomalous, outcome. This significantly improves accuracy, especially for loops, which are now mispredicted only on the final exit iteration, yielding an accuracy of $(N-1)/N$. Bimodal predictors can achieve accuracies around 90% and are considered a fundamental dynamic prediction technique.

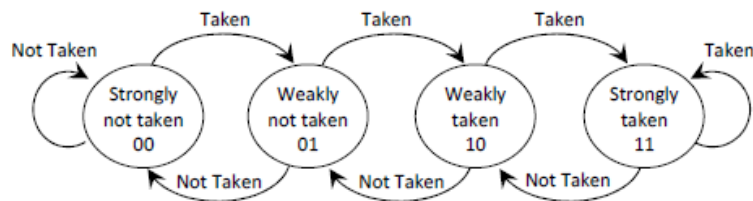


Figure 2: 2-bit predictor

2.3 Advanced Dynamic Prediction

While bimodal predictors significantly improve upon static methods, they only consider the past behavior of the individual branch being predicted. Further accuracy improvements were realized by recognizing that the outcome of a branch can be correlated with the outcomes of other recently executed branches (global history) or with longer patterns of its own past behavior (local history).

2.3.1 Two-Level Adaptive Predictors

They employ two levels of history information. The first level records a pattern of recent branch outcomes (either global history from all branches or local history specific to a branch address). This history pattern is then used to index into a second-level structure, typically a Pattern History Table (PHT) containing arrays of predictors (like 2-bit saturating counters). This allows the prediction for a given branch to adapt based on the context provided by preceding branch outcomes. Gshare predictor is an example of a Two-Level Adaptive Predictor.

GShare records the outcomes of the last 'k' executed branches, with the lower bits of the current branch's PC address. This combination, usually achieved through a simple hash function like XOR, is used to index into a single large PHT of 2-bit counters. By incorporating the PC address into the index, Gshare aims to reduce aliasing, where different branches with the same global history pattern map to the same counter entry, potentially interfering with each other's predictions.

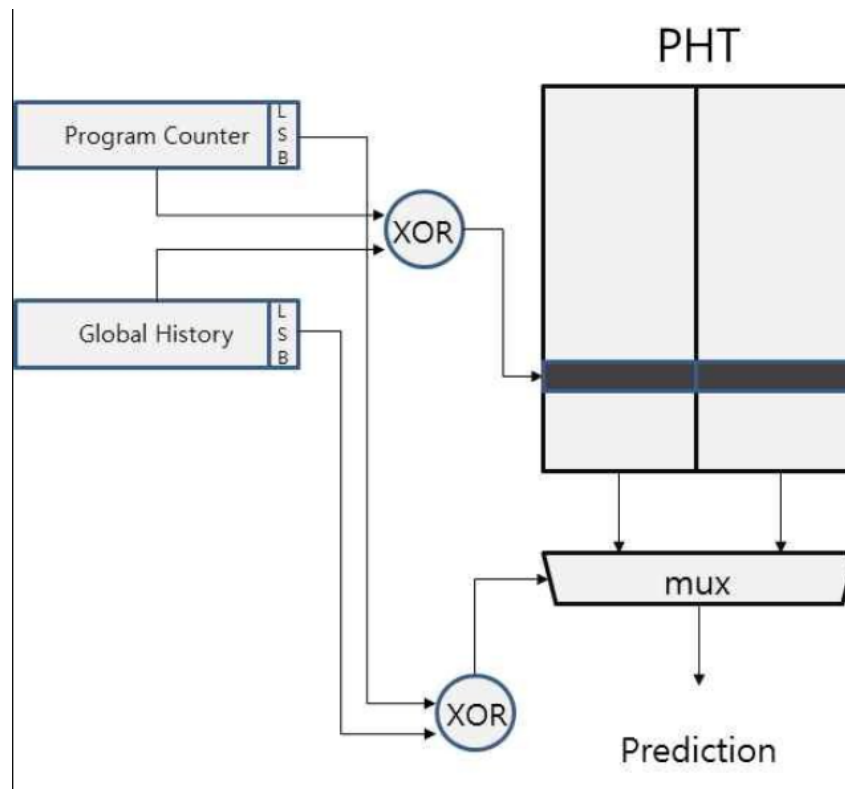


Figure 3: GShare predictor

2.4 State-of-the-Art: TAGE-SC-L

This represents the state-of-the-art in conventional (non-deep-learning) branch prediction, often winning branch prediction championships and serving as a baseline for comparison. It is a hybrid predictor.

The TAGE-SC-L design philosophy exemplifies achieving high accuracy through hybridization and specialization: combining a powerful, general-purpose predictor (TAGE) with specialized components (SC, L) that target its specific weaknesses. Its performance is highly dependent on the allocated storage budget. Despite its effectiveness, TAGE-SC-L still struggles with certain H2P branches and large-footprint server workloads.

2.4.1 TAGE (Tagged GEometric history length)

The core component uses multiple predictor tables, each indexed by a hash combining the branch PC and a snippet of the global branch/path history. The history lengths used for indexing these tables increase geometrically (e.g., $L(i) \approx \alpha^{(i-1)} * L(1)$), allowing the predictor to capture correlations over vastly different time scales. Each table entry contains a prediction counter (typically 3-bit), a partial tag (derived from history/PC) for reducing aliasing, and a "useful" counter (u-bit) to manage entry allocation and replacement. Prediction is usually based on the entry in the table with the longest history length that matches the current context (tag match). TAGE is highly storage-efficient for leveraging very long histories. It includes a simple bimodal predictor as a base case (T0).

2.4.2 Statistical Corrector (SC)

This component aims to correct systematic errors made by the main TAGE predictor. It learns, based on various context features (TAGE's prediction, TAGE's confidence level, global/local histories, path history, other microarchitectural events like IMLI counters), when TAGE is likely to be wrong and overrides its prediction. The SC itself is often implemented using perceptrons or multiple smaller tables.

2.4.3 Loop Predictor (L)

A small, specialized component designed to accurately predict the behavior of simple, regular loops, especially those with large iteration counts or long bodies, which might otherwise cause mispredictions in TAGE on loop exit.

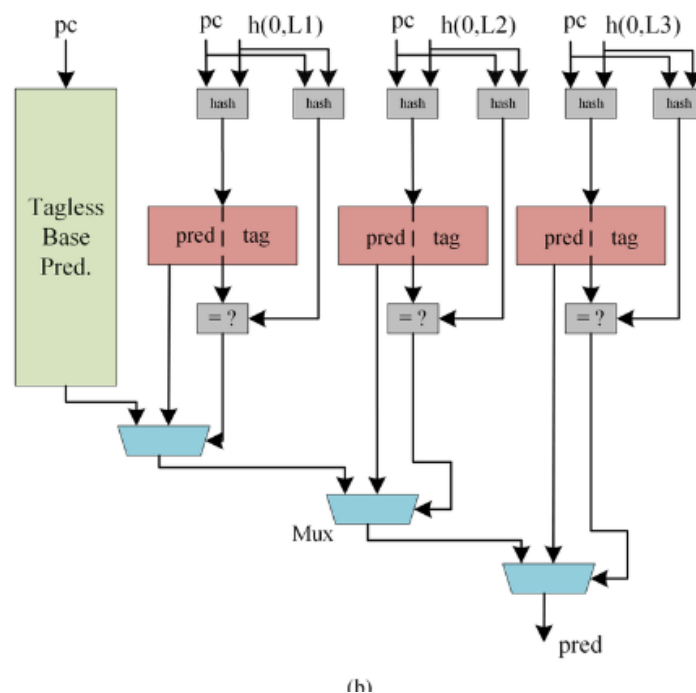
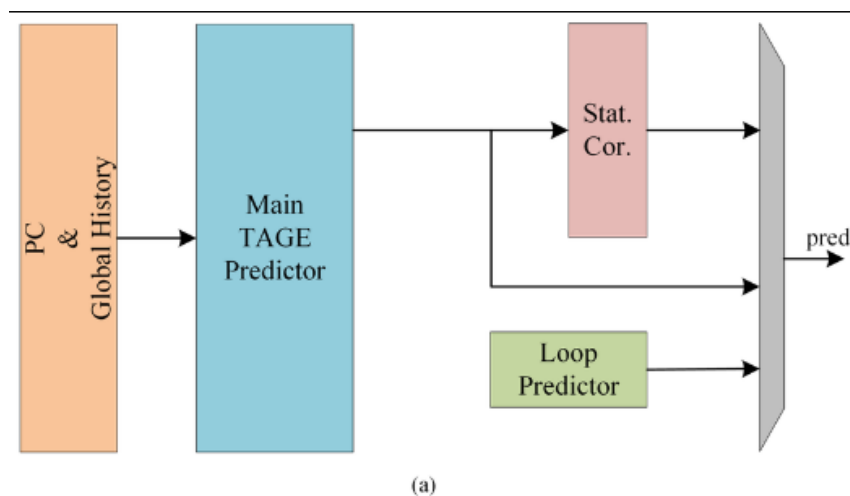


Figure 4: TAGE-SC-L predictor

2.5 Machine/Deep Learning Architectures

Despite the sophistication of traditional dynamic branch predictors, their accuracy is not perfect, and the quest for higher performance motivated exploring new techniques, leading to the application of machine learning (ML) concepts.

While advanced traditional predictors like TAGE (TAGged GEometric history length) achieve very high accuracy for the vast majority of branches often exceeding 99% accuracy for 99% of static branch instructions in benchmark suites like SPEC a small subset of branches remains stubbornly difficult to predict. These "hard-to-predict" (H2P) branches, despite being relatively few, can account for a disproportionately large fraction of the total mispredictions and significantly impact overall processor performance.

2.5.1 Perceptron Predictor

This approach applied one of the simplest forms of neural networks, the perceptron, treating branch prediction as a binary classification problem: classifying a branch as either Taken or Not Taken.

The predictor maintains a table of perceptrons, where each perceptron is essentially a vector of small, signed integer weights. This table is indexed by hashing the address (PC) of the branch instruction. The inputs to the selected perceptron are the bits from the Global History Register (GHR), typically represented in bipolar form (e.g., +1 for Taken, -1 for Not Taken), along with a constant bias input of +1.

The perceptron computes its output (y) by calculating the dot product of its weight vector (w) and the input vector (x , composed of history bits and the bias): $y = w_0 + \sum_{i=1}^n (x_i * w_i)$ for $i=1$ to history length. The prediction is then made based on the sign of this output value ' y '. If ' y ' is non-negative, the branch is predicted Taken; if ' y ' is negative, it is predicted Not Taken.

2.5.2 Tarsa's Predictor

A simple predictor using convolutional networks was introduced by Stephen Tarsa. This relatively straightforward neural network takes as input a one-hot encoding of a history of branch events specifically pairs of (Branch Program Counter - Branch Outcome) of length 200. It then employs a convolutional layer to detect correlations among the branches.

Just as in image analysis a convolutional neural network identifies patterns in images, here it can similarly identify, within a history of branches, the significant branches that matter for making a prediction. Once the pattern matching is complete, the result is fed into a linear layer, which selects the final prediction.

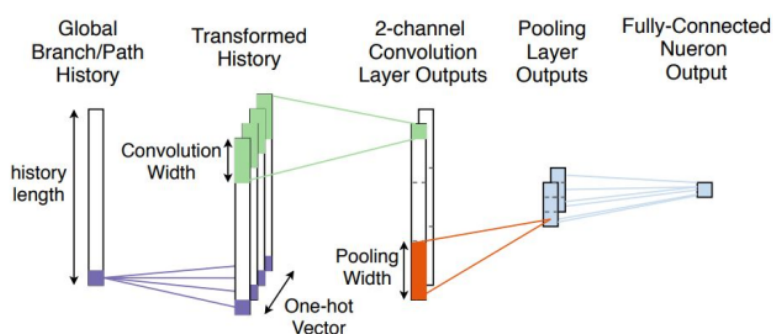


Figure 5: Tarsa's Predictor

2.5.3 BranchNet

A more advanced predictor based on Tarsa’s idea is the implementation of BranchNet. The main difference compared to the original work is that larger histories create issues at the final (linear) layer due to the sheer volume of data. To leverage longer histories and address this problem, BranchNet employs sum pooling layers, which act as a sampling mechanism, enabling the processing of extended histories along with more branch PC bits by the final linear layer. Its architecture is inspired by traditional predictors and uses geometrically increasing history lengths as input. As shown in figure below, for each history slice we first apply embedding-based encoding, then a convolutional network to detect patterns, followed by sum pooling, so that only information from the relevant branches passes to subsequent layers. Thus, in the final linear layer just as in Tarsa’s model, only the relevant branch information arrives, despite the large history length used. This architecture uses 582 history entries (instead of 200) while encoding the 11 least significant bits of the branch program counter.

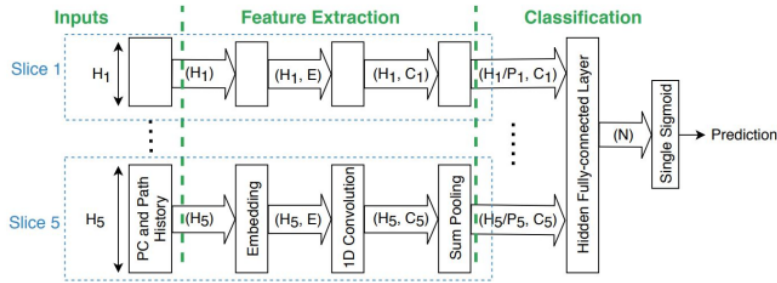


Figure 6: BranchNet

2.6 Our Architecture: *BranchNet-LSTM*

We propose an architecture inspired by BranchNet, in which each sum-pooling layer is replaced by a Long Short Term Memory (LSTM) layer, where the rest of the architecture remains intact. We follow the same Slice/Layer design and we keep the same input sequences. Our neural network architecture is composed of five distinct slices, each tailored with unique configurations to capture diverse patterns in the input data. Each slice differs in two key aspects: the kernel sizes of the convolutional layers and the history lengths of the five input sequences. This variation allows each slice to specialize in extracting features at different scales and temporal depths. Upon processing the input data, the outputs from all five slices are concatenated, forming a comprehensive feature representation that contains the specialized information from each slice. This concatenated feature vector is then passed through subsequent layers, enabling the model to leverage the rich, multi-scale information for improved performance in branch prediction tasks.

2.6.1 Input Sequences

Our neural network model processes five distinct input sequences, each representing the same trace history but with varying lengths: 42, 78, 150, 294, and 582. These sequences capture branch correlations across different history depths, enabling the model to learn both short-term and long-term dependencies in branch behavior.

Each sequence is composed of pairs (PC_i, b_i) , where PC_i denotes the program counter of the i -th branch, and b_i is the corresponding branch outcome: $b_i = 1$ if the branch was taken,

and $b_i = 0$ if not taken. For instance, a segment of the sequence might look like:

$$[(PC_A, 0), (PC_X, 1), (PC_A, 0), (PC_X, 0), (PC_B, 1), \dots]$$

This structure allows the model to detect recent, fine-grained patterns through shorter history slices, while longer slices enable the identification of deeper, more complex correlations that span hundreds of branches. By processing these sequences, the model effectively captures diverse branch correlations across varying history lengths, enhancing its predictive accuracy.

2.6.2 Embedding

The embedding layer functions as a lookup table that maps hashed numerical indices to dense, continuous vectors, the embeddings. The embeddings are trainable parameters of the network, adjusted during the training process to capture meaningful relationships between input indices. By learning to associate similar indices with similar embedding vectors, the model effectively captures semantic or relational information inherent in the data. Importantly, embeddings are compact representations, requiring minimal storage space per entry, which enhances the efficiency of the model. This approach contrasts with traditional one-hot encoding, which results in sparse, high-dimensional vectors, thereby improving computational efficiency and model performance.

Εντολές	10 Λιγότερα σημαντικά ψηφία PC	Αποτέλεσμα για Not- Taken	Αποτέλεσμα για Taken
400587: jle 400599	..0110000111	14	15
40058a: ja 400599	..0110001010	10	11
400bcd: je 400599	..1111001101	154	155
450307: jle 450a03	..0000000111	14	15

Figure 7: Example with History Hashing. We keep the 7 least significant bits of the program counter of the branch.

2.6.3 Conv1D

In our branch prediction model, the Convolutional Neural Network (CNN) layer serves as a very important component for feature extraction. Due to the encoding of the Embeddings it is observed that CNN layers can be used to exploit their 'spatial' behavior. This encoding allows the CNN to effectively capture local dependencies and patterns within the branch history. By applying convolutional filters, the network identifies correlations between branches, making more accurate predictions. Additionally, CNNs are better supported for hardware implementation compared to Long Short-Term Memory (LSTM) networks. Their architecture, characterized by local connectivity and weight sharing, aligns well with the parallel processing capabilities of hardware fabrics, leading to efficient deployment and execution.

2.6.4 AvgPool1D

The Average Pooling layer serves as a crucial component in Convolutional Neural Networks (CNNs), used for dimensionality reduction. By aggregating the average of values within a defined window across the feature map, it effectively reduces the spatial dimensions, thereby decreasing the number of parameters and computational load in the network. This reduction not

only enhances the efficiency of the model but also contributes to its robustness. The averaging process helps in summarizing features, making the network less sensitive to small distortions or translations in the input data. Consequently, the Average Pooling layer provides a generalized representation of the features within the pooling window, capturing essential patterns while mitigating the impact of noise and minor variations.

2.6.5 LSTM

Unlike sum-pooling, which collapses a sequence of branch histories into a single scalar and reduces the network's feature dimensionality, LSTMs preserve the full temporal representation and excel at modeling sequential dependencies. The LSTM's gates allow the model to learn, for each branch, how much historical information to retain or forget, rather than enforcing an equal weighted sum. This is especially beneficial when certain history patterns carry greater predictive significance. Moreover, by maintaining the original ordering of Taken (T) and Not-Taken (NT) results, LSTMs capture execution patterns that sum-pooling wouldn't be able to capture. Although the introduction of LSTM layers increases both the number of parameters and the computational complexity, being aware of the order of feature representation leads to a more accurate branch prediction performance. Furthermore, LSTM layers excel at modeling long-range dependencies within each branch history, enabling the network to recognize extended patterns that sum-pooling would obscure. Their capacity to capture such long-term relationships yields a more informative feature representation for prediction.

2.6.6 Fully Connected Dense Layers and Sigmoid Activation

Following the concatenation, two fully connected (dense) layers are incorporated sequentially to enable the model to learn intricate combinations of features—capabilities that convolutional and recurrent layers alone may not fully capture. The dense layers facilitate this by providing each neuron with direct connections to all neurons in the preceding layer, thereby allowing the model to learn complex, non-linear relationships between features. This architecture is particularly beneficial for tasks requiring high-level reasoning and decision-making. To conclude the network, a sigmoid activation function is applied in the output layer. This function maps the output to a probability value between 0 and 1, making it ideal for binary classification tasks such as predicting whether a branch is taken (T) or not taken (NT). The sigmoid function's output can be interpreted as the likelihood of the positive class, simplifying the decision-making process in binary classification scenarios.

Our Neural Network architecture is displayed below broken into two parts:

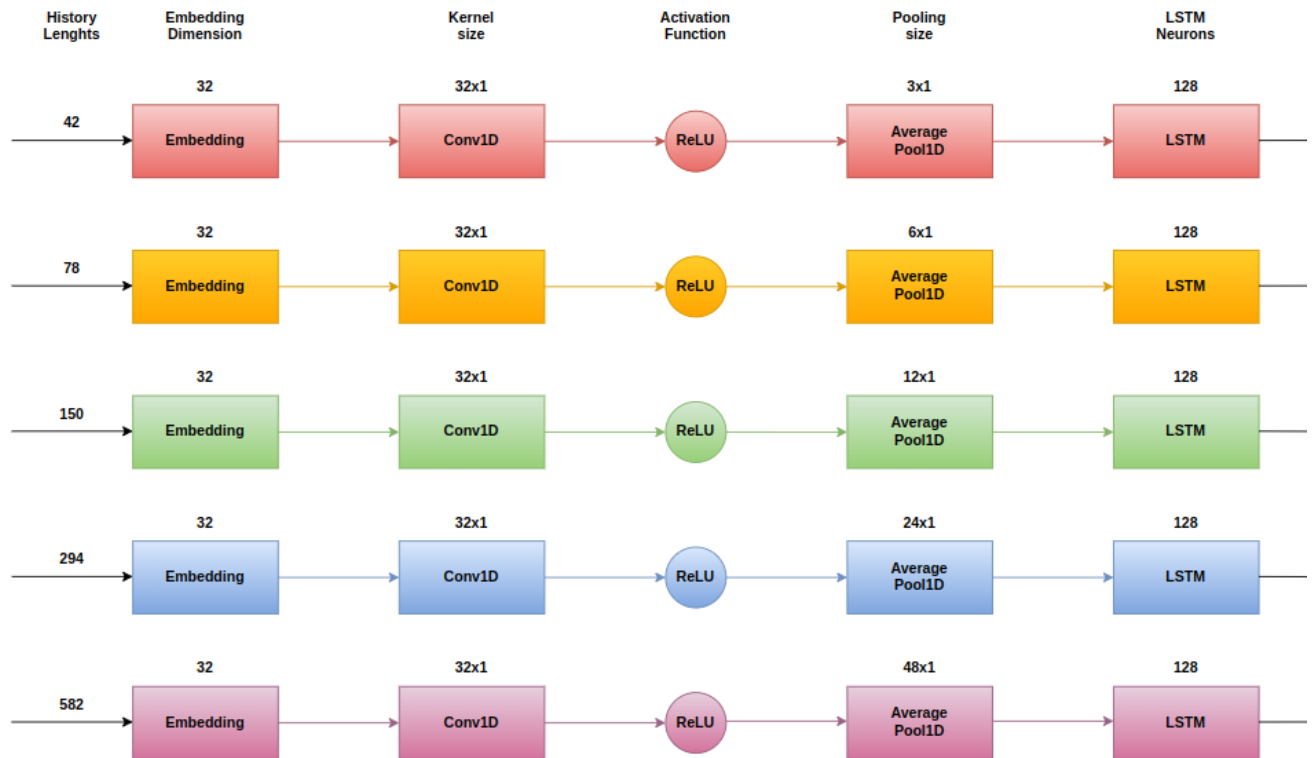


Figure 8: Our Architecture Part-1

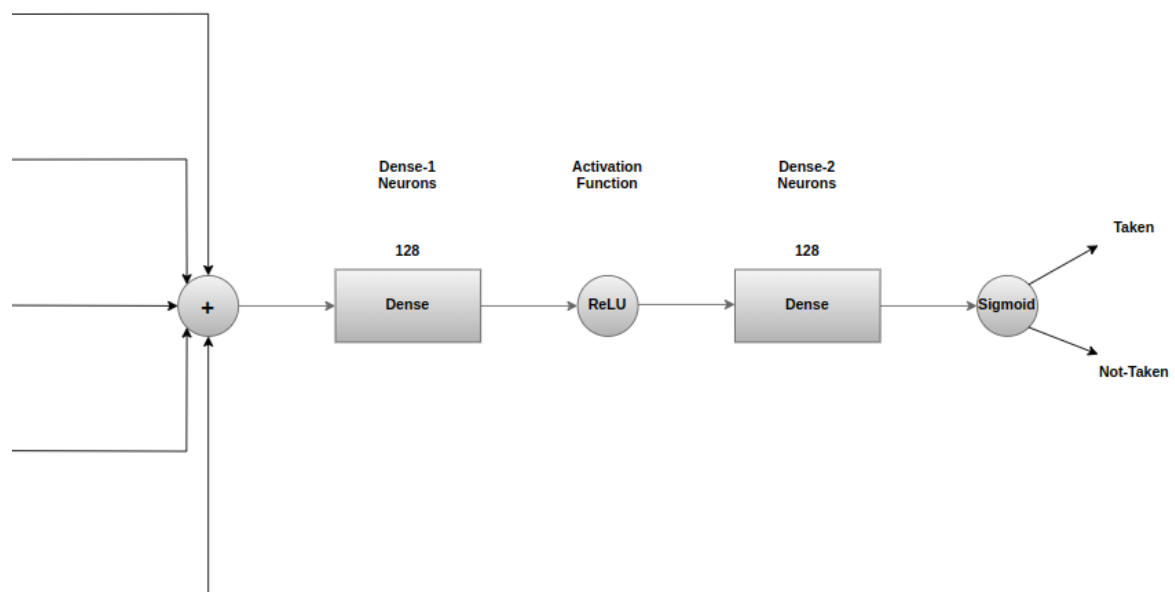


Figure 9: Our Architecture Part-2

3 HLS4ML

The `hls4ml` library is an open-source software framework designed to facilitate the deployment of machine learning (ML) models on field-programmable gate arrays (FPGAs), targeting low-latency and low-power edge applications. By taking a trained neural network model as input, `hls4ml` generates C/C++ code that can be transpiled into FPGA firmware using a high-level synthesis (HLS) tool.

3.1 Challenges

3.1.1 Model Complexity

Our model had a very complex structure and couldn't be implemented with `hls4ml` directly. The model would have to be simplified first to be implemented on hardware.

3.1.2 Framework Compatibility

The original model was developed using Pytorch. Though, the limited support `hls4ml` provides for Pytorch and especially no support for the Embedding layer lead us to converting the model to tensorflow.

3.1.3 Memory Constraints

Both the 5-layer and the smallest 3-layer model exceeded the available memory resources during the synthesis, leading to a failure in building the bitstream for the FPGA.

3.1.4 Training without Batch size

Training the model and then loading it in the `hls4ml` format led to discrepancies in the simulation results and errors. We had to adjust by training without the batch size, which was very slow for the performance of our model.

3.2 Process

3.2.1 Model Training & Evaluation

We trained once again our simplified tensorflow neural network model and afterwards we applied evaluation to access its performance.

3.2.2 Quantization Configuration

To prepare the model for FPGA deployment, we configured the quantization settings using `hls4ml.utils.config_from_keras_model`. This function generated a configuration dictionary, allowing us to specify precision settings for weights and activations.

3.2.3 Model Conversion

Using the generated configuration, we converted the Keras model into an HLS-compatible model with `hls4ml.converters.convert_from_keras_model`. This step translated the model into a format suitable for FPGA synthesis.

3.2.4 Model Compilation

We compiled the HLS model using `hls4ml.model.compile`, which processed the model's configuration and prepared it for synthesis, ensuring compatibility with the target FPGA.

3.2.5 Model Building

The synthesized model would have then been built using `hls4ml.model.build`. This function generated the necessary files for FPGA deployment, including the bitstream. We would utilize the `csim=True` parameter to perform C simulation, verifying the model's functionality before hardware implementation.

3.2.6 Performance Reporting

After building the model, we would have assessed its performance by parsing the Vivado HLS report with `hls4ml.report.read_vivado_report`. This would provide insights into resource utilization, latency, etc.

3.2.7 Bitstream Generation

Finally, we would generate the FPGA bitstream, preparing the model for deployment on the PYNQ-Z2 board. This step would involve finalizing the design and ensuring that all configurations were correctly implemented for hardware execution.

4 Dataset Generation

4.1 Computer Architecture Simulators

Simulators are essential tools in computer architecture because they allow researchers, educators, and students to explore and analyze processor designs without the cost or complexity of building real hardware. They provide a safe and flexible environment to test ideas, evaluate performance, and understand how different microarchitectural components interact. Especially as processor designs grow more complex, simulators like ChampSim, Gem5, and Sniper offer modular, customizable platforms that lower the barrier to entry, making it easier to experiment and learn in the field.

4.1.1 ChampSim

For this project, we used the [ChampSim](#) simulator, an open-source trace-based simulator developed and maintained at Texas A&M University. ChampSim is widely used across the computer architecture community, even in competition for branch prediction or data prefetching ([DPC3](#)).

For our project, we used a modified version of the ChampSim simulator from [4]. This version allows us to use the simulator both for the generation of the dataset, as well as for the evaluation of the trained neural networks.

4.1.2 Traces

The ChampSim traces are files that record the sequence of memory and instruction events generated by running a program, specifically formatted for use with the ChampSim microarchitectural simulator. These traces are essential inputs for ChampSim, which is a trace-based simulator used for research and study of CPU microarchitecture, such as cache replacement policies, branch prediction, and prefetching.

The simulator provides us with a tracing tool (`champsim_tracer.so`), that can be used alongside the Intel Pin tool to generate champSim-specific traces. After building the provided tracer, your target program can be run under the Pin tool and the tracer. During this step, the tool gives you some useful options, such as the ability to skip a certain number of instructions and also to specify the number of instructions that will be recorded.

In our case, we chose to use already recorded traces from the [3rd Data Prefetching Competition \(DPC3\)](#). These traces were generated using the SPEC CPU 2017 benchmark suite and are available online ([here](#)).

4.2 Hard-to-Predict (H2P) Branches

The term Hard-to-Predict branches (H2P) was used by Tarsa et al [3] to describe a class of branches that state-of-the-art runtime branch predictors like TAGE struggle to predict accurately because these branches are plagued by noisy and complex correlations in the long global branch history. However, according to Tarsa, such complex, noisy correlations in branch history can be better predicted by sophisticated offline-trained models like CNNs.

To identify and categorize a branch as hard-to-predict, we follow the same approach as Tarsa [3]: within each 30M-instruction slice of every workload, we screen branches and identify as H2Ps those that

1. have less than 99% prediction accuracy under TAGE-SC-L 8KB
2. execute at least 15,000 times
3. generate at least 1,000 mispredictions

4.2.1 Identifying H2P branches

In order to identify the H2P branches for a given workload, we used, as mentioned above, the modified ChampSim simulator. Here, we give as input to the simulator the trace file that we want to simulate alongside the parameters for categorizing a branch as hard-to-predict (occurrences, misspredictions, accuracy). The simulator starts its execution, keeps statistics for each branch, and later categorizes it as H2P if it meets the criteria mentioned above.

In our case, we had access to 5 different traces for each benchmark (i.e. *leela*). These five traces were different in terms of input as well as the number of instructions recorded. We executed the simulation for each trace and kept the H2P branches of each trace. Then, like [2], we choose to keep only the H2P branches that appear in at least 40% of the traces. This way, we keep only branches that are independent of the input and will likely appear again in future program executions.

4.3 Dataset Structure

5 Model Training

As mentioned above, our dataset includes many H2P branches; thus, we train one neural network for each H2P branch. For the training of the model, we followed the approach from BranchNet [1]. We trained our model in 3 phases, where in each phase we reduced the learning rate by a factor of 0.2. As for the optimizer, we used Adam and MSE for the loss function.

5.1 Training Data

For the training of our models, as discussed in Section 4.1.2, we used traces from the DPC3 competition. In the data provided by DPC3, there are approximately 5 traces for each SPEC CPU benchmark. Here, we chose to use the [Leela](#) benchmark, since according to BranchNet [1], Leela is amongst the benchmarks that benefit the most from the neural network predictors due to its hard-to-predict branches.

As mentioned, the DPC3 provides 5 different traces for the Leela benchmark. For each of the traces, we performed a simulation using the ChampSim simulator and identified all the H2P branches. We followed the approach described in Section 4.2.1 to create our dataset.

From the provided Leela traces, we identified 45 H2P branches. Since these branches appear at least in 40% of the provided traces, they are the perfect candidates for neural networks to be trained on since there is a good chance that they are not input specific and will probably occur again in future executions of the benchmarks.

After gathering the dataset and identifying the H2P branches, we trained one neural network for each branch. We followed this approach for both the BranchNet neural network as well as our implementation.

6 Model Evaluation

For the evaluation of the neural network models, we needed an architecture as the one shown in Figure 10. For this, we used the ChampSim provided by [4], in order to evaluate the performance of our trained networks through simulations. According to [4], there are two evaluation methodologies:

1. Online integration of the neural predictor during simulation.
2. Offline prediction using log files.

While the first provides the most realistic evaluation of predictor latency and integration, it introduces significant overhead. Each forward pass can be computationally expensive, and performing it during simulation leads to long execution times, often stretching over multiple days, even on multi-core systems. To address this bottleneck, [4] developed the second methodology. There, the neural network generates predictions ahead of time using historical branch data (from the traces) and stores them in log files. Then, during the simulation, when an H2P branch appears, its prediction can be simply read from the logs, eliminating the need for online model inference.

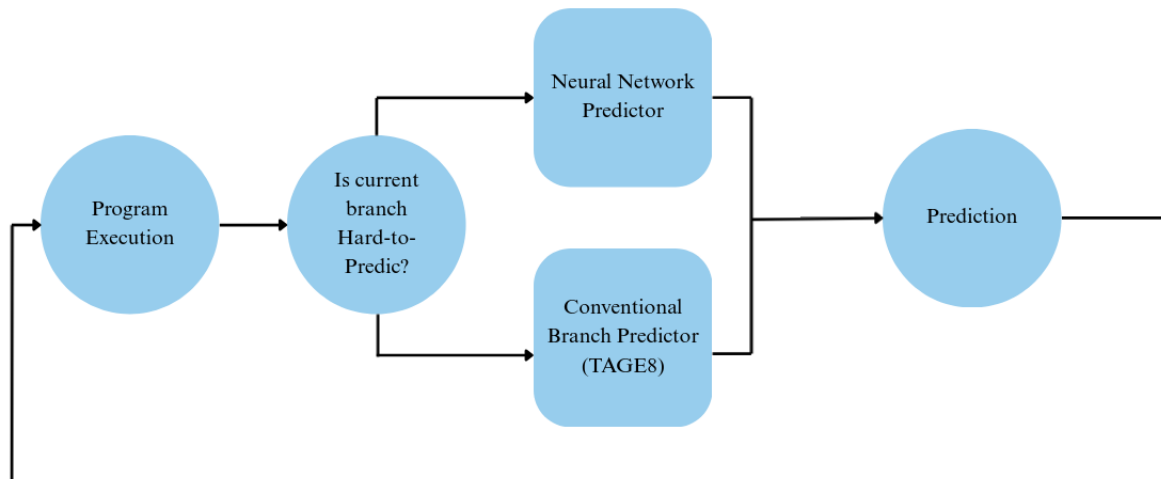


Figure 10: Branch Prediction with Neural Network Predictor Helpers

6.1 Results for different predictors

Table 1 shows the performance of different conventional branch predictors against the trace used for evaluation. The perfect predictor refers to a TAGE-SC-L 64KB predictor, with the ability to predict all 45 H2P branches correctly. It gives us the upper limit that our neural network should aim for.

Predictor	Accuracy (%)	Difference from Perfect (%)
Bimodal	87.28	-10.82
TAGE8	92.49	-5.61
TAGE64	93.83	-4.27
TAGE64-BranchNet-LSTM	95.45	-2.65
Perfect	98.10	0.00

Table 1: Comparison of predictor accuracies

Our neural network model, integrating the TAGE64-BranchNet-LSTM predictor, achieved an accuracy of 95.45%, reducing the gap from the theoretical perfect predictor to just 2.65% (from the initial 4.27% of the TAGE64 predictor) and improving on the accuracy of the standalone TAGE64 predictor. This improvement demonstrates the effectiveness of combining the state-of-the-art branch predictor with a deep learning model for enhanced performance.

6.2 Comparing our model with the Original BranchNet

When compared with the original BranchNet model, our model demonstrated significant improvements across all 45 branches evaluated. The total number of correct predictions increased by 23,816,042, representing a 194.91% enhancement in overall accuracy. This substantial gain underscores the effectiveness of our approach in refining branch prediction accuracy.

However, because these hard-to-predict (H2P) branches represent only a small fraction of the total branches encountered during simulation, the overall accuracy metric remains relatively unchanged (with the TAGE64-branchNet predictor achieving an overall accuracy of 95.1%).

The vast majority of branches are easily predicted by traditional predictors such as TAGE-SC-L, which dominate the overall accuracy calculation. As a result, even a major improvement in H2P prediction performance, such as tripling the number of correct predictions, can translate to only a marginal increase in global prediction accuracy. This highlights the importance of evaluating branch predictors not only on overall accuracy but also on their performance specifically over H2P branches, where improvements have the greatest impact on reducing misprediction penalties and enhancing processor efficiency.

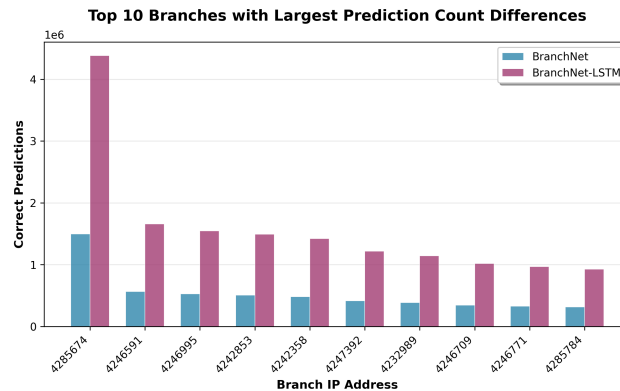


Figure 11: Comparing the top 10 branches that have the biggest differences.

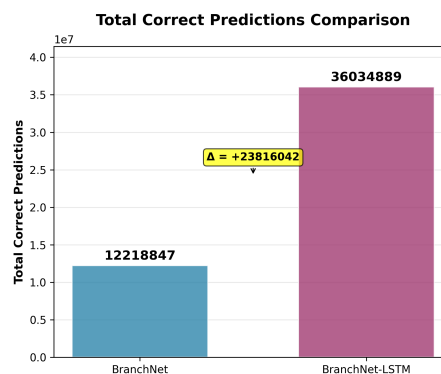


Figure 12: Total correct predictions for BranchNet vs our model

Metric	Value
# Branches Improved	45
Total Correct Predictions	23,816,042
Overall Accuracy Increase	194.91%

Table 2: Performance Improvement Over Original BranchNet

7 Future Work

To further evaluate and enhance the predictor, future work will involve assessing its impact on the underlying microarchitecture. This includes examining factors such as instructions per cycle (IPC), pipeline stalls, and execution latency to understand how the predictor interacts with

complex core designs in simulation environments. Expanding the evaluation to a broader set of benchmarks, such as mcf, deepsjeng, and xz, will also provide deeper insights into the performance of the predictor across diverse workloads. Additionally, training on a larger and more diverse set of execution traces could improve the predictor's ability to identify hard-to-predict (H2P) branches that are independent of specific input data. From a hardware perspective, implementing the neural branch predictor on an FPGA platform is also another path. Optimizations such as quantization and pruning will be investigated to reduce area, latency, and power consumption while maintaining prediction accuracy. Focusing on the predictor's size is one of the most important steps of future work, since deploying such a large number of large networks on hardware, especially alongside a CPU, is not feasible, and smaller, more compact designs should be investigated.

8 References

- [1] Zangeneh, Siavash & Pruett, Stephen & Lym, Sangkug & Patt, Yale. (2020). BranchNet: A Convolutional Neural Network to Predict Hard-To-Predict Branches. 118-130. 10.1109/MICRO50266.2020.00022.
- [2] Gober, N., Chacon, G., Wang, L., Gratz, P. V., Jimenez, D. A., Teran, E., Pugsley, S., & Kim, J. (2022). The Championship Simulator: Architectural Simulation for Education and Competition. ArXiv. <https://arxiv.org/abs/2210.14324>
- [3] Lin, C., & Tarsa, S. J. (2019). Branch Prediction Is Not a Solved Problem: Measurements, Opportunities, and Future Directions. ArXiv. <https://doi.org/10.1109/IISWC47752.2019.9042108>
- [4] Vontzalidis, Aristotelis K. (n.d.). Branch Instruction Prediction Using Artificial Neural Networks: Diploma Thesis. Supervisor: Dionysios Pneumatikatos, Professor. School of Electrical and Computer Engineering, National Technical University of Athens. Available at: <https://github.com/aristotelis96/thesis-branch-predicition-ml>